

RELAZIONE PROGETTO UNO

Il progetto è stato sviluppato da *Florea Alexandru Cosmin* (M. 2241398) e *Giorgini Massimo* (M. 2234123) tramite l'utilizzo di Java 21 e JavaFX per la l'interfaccia grafica con l'obiettivo di soddisfare tutti i requisiti previsti per il gruppo di due persone. Per motivi organizzativi lo sviluppo è stato portato avanti in parallelo sia su Eclipse, che su Visual Studio Code e per questo motivo sono presenti due launcher distinti, entrambi perfettamente funzionanti e spiegati meglio nel file README.md

1. DESCRIZIONE GENERALE

Il progetto permette di gestire partite per un numero variabile da 2 a 6 giocatori, supportando la possibilità di scegliere tra partite di soli giocatori umani, partite miste tra umani e bot, e simulazioni esclusivamente tra bot.

All'avvio di ogni partita, il mazzo viene generato rispettando la disposizione delle 108 carte del regolamento ufficiale:

- 76 carte numeriche colorate suddivise nei quattro colori classici e per ogni colore è presente uno solo 0 mentre i numeri da 1 a 9 sono presenti due volte.
- 24 carte azione colorate divise tra i quattro colori di cui due carte Skip, due Reverse e due Draw Two.
- 8 carte Wild tra quattro carte Wild semplici e quattro carte Wild Draw Four.

Per modellare le carte abbiamo seguito una semplice struttura gerarchica:

- Card.java (Classe Astratta): Rappresenta una carta generica da gioco e definisce campi e metodi fondamentali di ogni carta:
 - type (tipo CardType, protected e final): definisce il tipo di carta
 - originalColor (tipo CardColor, protected e final): rappresenta il colore originale della carta
 - chosenColor (tipo CardColor, private): memorizza il colore scelto dal giocatore dopo aver giocato una carta Wild (resta null per le carte normali)
 - drawn (boolean, private): traccia se la carta è stata appena pescata dal mazzo durante il turno attuale

Tra i metodi c'è `getActiveColor()` che restituisce il colore attivo in gioco restituendo `chosenColor` per i jolly e `originalColor` per le altre carte, mentre `setChosenColor(CardColor color)` imposta il colore per i Wild lanciando un'eccezione se chiamato su altre carte. `isPlayableOn(Card topCard)` verifica la compatibilità con l'ultima carta giocata (sempre vero per i Wild, oppure se combaciano colore attivo o tipo) e infine il metodo astratto `getPointsValue()` che delega alle sottoclassi il determinare il punteggio di fine round.

- NumberCard.java (Estende Card): È la classe concreta per le carte numerate (da 0 a 9). Introduce il campo privato e immutabile value per il valore numerico. Il costruttore valida che il colore sia effettivo (diverso da NONE) e che la cifra sia compresa tra 0 e 9. Implementa getPointsValue() restituendo direttamente il valore facciale della carta, ed effettua l'override di isPlayableOn() consentendo la giocata anche se la carta a terra ha lo stesso numero, a prescindere dal colore.
- ActionCard.java (Estende Card): Rappresenta le carte speciali colorate (Skip, Reverse e Draw Two) con la costante POINTS_VALUE = 20. Il metodo getPointsValue() restituisce sempre 20 punti e permette la giocata se combacia il colore attivo o se è un'azione dello stesso tipo.
- WildCard.java (Estende Card): con la costante POINTS_VALUE = 50 e include un booleano canCauseChallenge (vero solo per i +4). È giocabile su qualsiasi carta (quindi isPlayableOn() è sempre true) e include i metodi getter e setter per canCauseChallenge per evitare che la carta una volta scartata, continui a generare sfide non dovute nei turni successivi.
- CardColor.java (Enum): Definisce i colori (RED, BLUE, GREEN, YELLOW) più il valore NONE per i Wild non ancora giocati ed ha metodi di come isRealColor() e getRandomColor().
- CardType.java (Enum): Definisce i vari tipi di carte (NUMBER, SKIP, REVERSE, DRAW_TWO, WILD, WILD_DRAW_FOUR) usando metodi molto richiamati come isWild() e isColoredAction().

Per gestire l'insieme delle carte nei vari contesti di gioco abbiamo adottato questa gerarchia:

- CardCollection.java (classe astratta): Fornisce la base comune alla raccolta di carte tramite una ArrayList<Card> cardList. Tra i vari metodi ci sono getNumCards(), isEmpty(), deleteAllCards() e getAllCardsCopy(), quest'ultima che restituisce una copia della lista per evitare errori con l'interfaccia grafica.
- Deck.java (estende CardCollection): Rappresenta il mazzo di pesca composto inizialmente da 108 carte che nel momento della creazione genera tutte le carte necessarie tramite createDeck() e le mescola tramite shuffleDeck(). Ci son metodi per pescare la prima carta drawFromTopCard() o per pescare una quantità casuale di carte drawCardRandom(), usato per le smazzate iniziali o le penalità.
- DiscardPile.java (estende CardCollection): Gestisce la pila degli scarti e permette di ottenere la carta in cima con getTopCard(), aggiungere nuove carte con addCard() e trasferisce tutte le carte, tranne l'ultima, nel mazzo di pesca per rimescolarlo quando questo viene finito con moveToDeck().
- Hand.java (estende CardCollection): Rappresenta la mano privata di ogni giocatore e permette di aggiungere carte addCardToHand(), calcolare il punteggio totale della mano getHandScore() e giocare una carta trasferendola nella pila degli scarti playCard().

2. MODALITÀ DI PARTITA, REGOLE DI GIOCO E CALCOLO DEL PUNTEGGIO

Prima di avviare il gioco, l'utente può scegliere tra due modalità di partita:

- Partita Singola che si disputa un singolo round dove il primo giocatore che esaurisce le proprie carte vince la partita.
- Partita a Punti che adotta una struttura su più round, proseguendo fino a quando uno dei giocatori non riesce a raggiungere la soglia stabilita (di default a 500 punti, ma personalizzabile nel menù). Al termine di ogni round, i punti accumulati dal vincitore vengono sommati al suo punteggio complessivo e qualora nessuno abbia ancora raggiunto i punti necessari, l'engine invoca automaticamente `initializeRound()` per azzerare e rigenerare il mazzo, distribuendo 7 nuove carte a ciascun giocatore.

Invece i punteggi a fine round vengono assegnati come richiesti dalle regole di UNO:

- Carte Numeriche prendono un totale di punti esattamente corrispondente al loro valore nominale (da 0 a 9).
- Carte Azione (Skip, Reverse, Draw Two) 20 punti ciascuna.
- Carte Jolly (Wild, Wild Draw Four) 50 punti ciascuna.

Al termine della mano, il giocatore che è riuscito a giocare tutte le proprie carte ottiene la somma complessiva dei punti derivanti dalle carte rimaste in possesso di tutti gli avversari.

Al fine di rendere il gioco più istintivo e realistico possibile, abbiamo integrato un timer nell'interfaccia grafica per la dichiarazione dell'UNO:

- Nel momento in cui un partecipante gioca la penultima carta, si avvia un timer di 1 secondo e contemporaneamente sul lato destro dello schermo compare un pulsante rosso con la scritta 'UNO!'.
- Se l'utente preme il pulsante in tempo, lo stato interno del giocatore viene aggiornato a `UNOState.Safe`.
- Se invece il timer arriva a zero senza che sia stato premuto il pulsante, lo stato passa automaticamente a `UNOState.Unsafe`.
- Nel corso del turno successivo per 3 secondi, compare a schermo il pulsante arancione 'CONTESTA' e se un avversario decide di premere il pulsante, il trasgressore in `UNOState.Called` è costretto a pescare 2 carte dal mazzo, in caso contrario lo stato ritorna in sicurezza su `Safe`.

Secondo le regole, la giocata della Wild +4 è ritenuta lecita solo se chi la esegue non possiede nella propria mano altre carte del colore attivo come segue:

- Il `GameEngine` mantiene traccia costante del colore attivo tramite il campo dedicato `previousColor`.
- Non appena viene giocato un +4, il giocatore successivo vede comparire un pop up con le due opzioni 'Accetta' e 'Sfida'.
- Se il giocatore opta per la sfida, il metodo `WildDrawFourChallenge()` effettua una verifica sulla mano del giocatore che ha giocato la carta Wild +4 ed in base al caso:
 - Sfida vinta (Giocata illegale) nel caso in cui l'avversario possedesse effettivamente carte del colore precedente, lui stesso è costretto a pescare le 4 carte, lasciando il turno successivo invariato.

- Sfida persa (Giocata legale) se l'avversario ha agito correttamente e non possedeva il colore, lo sfidante subisce la penalità originaria di 4 carte più 2 carte aggiuntive per l'errore (per un totale di 6 carte) perdendo regolarmente il proprio turno di gioco.
- Player.java (Classe Astratta): rappresenta la superclasse per modellare sia gli utenti umani sia i bot, mantenendo attributi essenziali come playerName, playerId, playerType (HUMAN o BOT), unoState, i punteggi parziali e totali. Attraverso il metodo astratto playTurn(Card topDiscard) obbliga le sottoclassi a definire le specifiche strategie di scelta e giocata delle carte, utile principalmente nella definizione della strategia dei singoli bot.
- HumanPlayer.java (Estende Player): gestisce la logica dell'utente umano che interagisce con l'applicazione mediante l'interfaccia JavaFX e fa uso di una lista temporanea selectedCardsFromUI per memorizzare le carte selezionate tramite i click effettuati a schermo, mentre il metodo playTurn() passa questi elementi al motore di gioco per validare la giocata.
- GameMode.java: contiene la configurazione inerente la tipologia di partita (a punti oppure singola, soglia di punteggio da raggiungere e gestione dello stato di fine partita).
- GameAction.java e GameHistory.java: GameAction è pensato per registrare in dettaglio ogni mossa effettuata (autore, tipo di azione, carte coinvolte ed esito delle sfide), stampando in linguaggio naturale (esempio 'Giocatore 1 gioca RED_7 e grida UNO!'). GameHistory invece si occupa invece di conservare il tracciato cronologico di tutte le azioni registrate, rendendolo consultabile sia in console, sia sulla GUI a partita conclusa.
- GameEngine.java: coordina l'elenco dei partecipanti, monitora lo stato dei singoli mazzi, applica le sanzioni, risolve le sfide e calcola i punteggi al termine di ciascun round. Durante l'inizio, invoca firstDiscardCard() per trovare una carta numerica regolare con cui iniziare la pila degli scarti senza danneggiare il primo giocatore.

3. STACKING E NUMBER RUSH

Come richiesto per il gruppo da due studenti, abbiamo aggiunto due varianti del regolamento:

- Stacking delle Carte Pesca (di +2 e +4): Nel gioco standard quando si gioca una carta +2 o +4 il giocatore successivo deve obbligatoriamente pescare e saltare il turno, mentre attivando la variante dello Stacking la penalità viene invece accumulata nel campo pendingDrawPenalty gestito dal GameEngine. In questo modo il giocatore di turno ha la possibilità di difendersi rispondendo con un altro +2 (su un +2) o un +4 (su qualsiasi carta), sommando le carte per il giocatore successivo. Il primo che non risponde sarà costretto a pescare la somma totale accumulata (es. $2 + 2 + 4 = 8$ carte).

- Number Rush: Questa regola consente di scartare più carte numeriche con lo stesso valore in un'unica mossa combinata. Per essere valida, la prima carta scartata deve essere compatibile con la cima della pila degli scarti, mentre quelle successive devono semplicemente avere lo stesso numero con l'ultima carta che andrà a posizionarsi in cima agli scarti, imponendo il colore per il turno successivo.

4. PROFILI DEI BOT

Per il gruppo da due abbiamo deciso implementare come tre bot richiesti, il Random come da richiesta obbligatoria, più il Random e l'Aggressive:

1. Random: Con un approccio totalmente casuale, seleziona la mossa da eseguire tramite tra tutte le opzioni giocabili al momento.
2. Conservative: Preferisce una gestione prudente conservando le carte azione e i Wild per il fine della partita, cercando di liberarsi prima delle carte numeriche semplici.
3. Aggressive: Segue uno stile di gioco opposto al Conservative dando la priorità alle carte di pesca (+2 e +4) o a quelle azione (Skip e Reverse) usando le carte numeriche solo quando non ci sono alternative.

5. MODALITÀ SIMULAZIONE

Accedendo al menù è possibile lanciare una simulazione automatizzata e supportata dall'interfaccia grafica 'AVVIA SIMULAZIONE (Con Grafica)'. In questa modalità l'utente ha la possibilità di osservare le mani scoperte di ciascun bot e per rendere le dinamiche comprensibili, la sequenza dei turni viene cadenzata mediante un oggetto `PauseTransition` di `JavaFX` impostato su 1.5 secondi, consentendo di capire il comportamento dei vari bot in tempo reale.

Per raccogliere dati statistici su un gran numero di partite abbiamo creato 'SIMULAZIONE VELOCE (50 Partite)'. Considerato che l'esecuzione di 50 partite sono pesanti, abbiamo delegato l'elaborazione a un thread asincrono dedicato (`new Thread(() -> { ... }).start()`).

6. SALVATAGGIO E RIPRISTINO

Per la gestione del salvataggio e del caricamento della partita abbiamo usato la libreria di Google Gson. Trattandosi di classi astratte come `Card` e `Player`, la libreria Gson non è in grado di determinare autonomamente la classe concreta da riconoscere durante la lettura del file JSON. In `SaveManager` abbiamo fatto uso di `JsonDeserializer<T>` che in base alla tipologia specificata nel file JSON, crea l'istanza appropriata scegliendo tra `NumberCard`, `Wildcard` o `ActionCard`, allo stesso modo, valutando i campi `playerType` e `botProfile`, il

processo di deserializzazione ricostruisce l'oggetto corretto tra HumanPlayer, RandomBot, ConservativeBot o AggressiveBot.

SaveManager si occupa di effettuare la lettura e la scrittura del file di salvataggio uno_savegame.json sfruttando le classi FileWriter e FileReader con il metodo saveGame() salva in formato JSON. Il metodo loadGame() poi legge il contenuto del file, riconfigura gli oggetti e infine chiama engine.initializeDeckOnly() ripristinando la partita senza alterare le carte già presenti nelle mani dei giocatori caricati.

7. ARCHITETTURA DELLE CLASSI

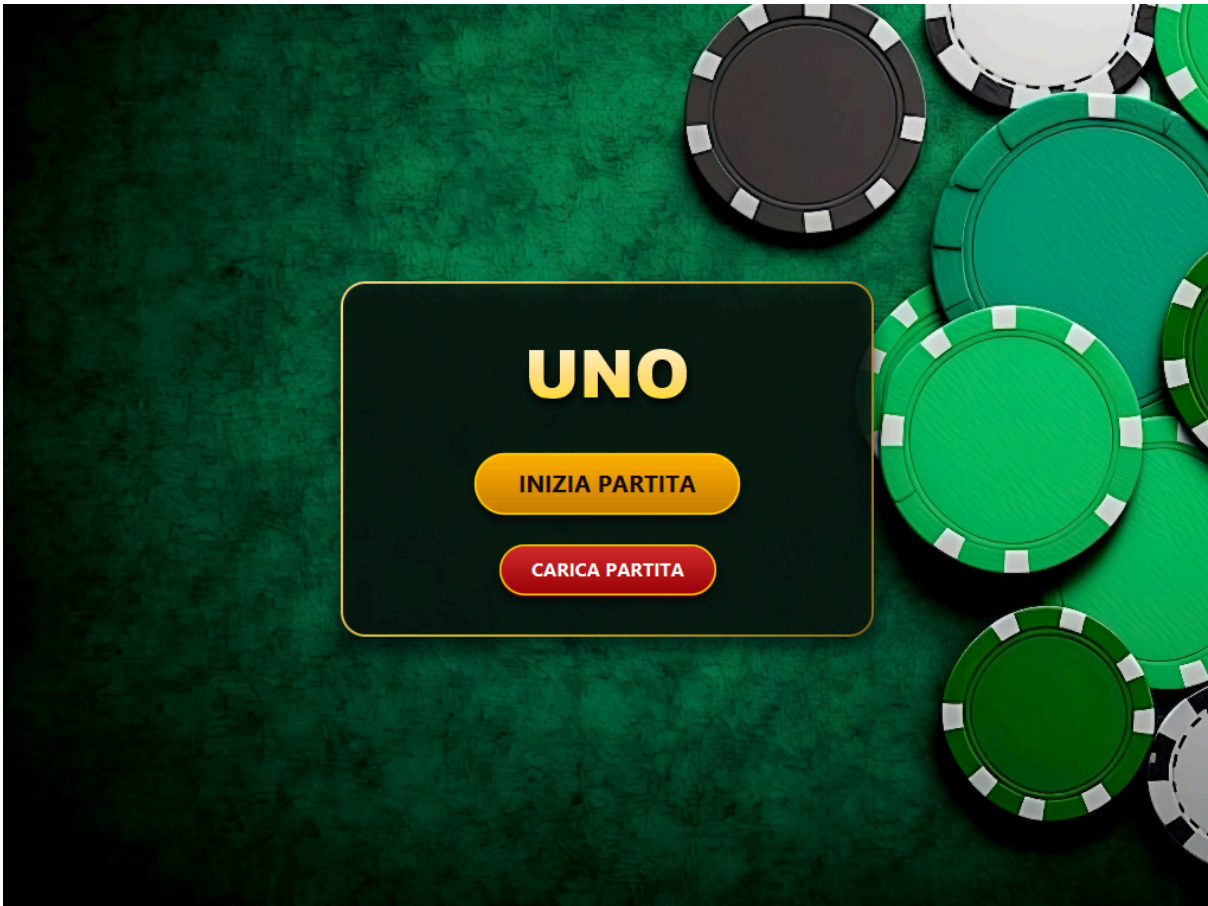
- Model: Ha la logica applicata e le regole del gioco (pacchetti card, deck, player, rules, game, simulation) risultando disaccoppiato dall'interfaccia grafica.
- View: Occupandosi solo dell'estetica dell'applicazione, la disposizione dei componenti visivi e la raccolta degli input utente tramite BoardView, CardView e MainApp.
- Controller: Si occupa degli eventi generati dalla View e occupandosi di aggiornarla di conseguenza con GameController.

Le varie classi che compongono il sistema sono organizzate in pacchetti coerenti:

- it.uniroma1.mdp.uno.model.card: Classi delle carte.
- it.uniroma1.mdp.uno.model.deck: Gestione del mazzo, pila degli scarti e mani.
- it.uniroma1.mdp.uno.model.player: Giocatori umani e bot.
- it.uniroma1.mdp.uno.model.rules: Configurazione delle varianti.
- it.uniroma1.mdp.uno.model.game: Motore di gioco, cronologia e turni.
- it.uniroma1.mdp.uno.model.simulation: Classi per le simulazioni.
- it.uniroma1.mdp.uno.save: Classi per il Salvataggio in JSON.
- it.uniroma1.mdp.uno.view: Interfaccia utente con JavaFX.
- it.uniroma1.mdp.uno.controller: Gestione degli eventi.

8. PROGETTAZIONE DELL'INTERFACCIA GRAFICA UTENTE (GUI JAVA FX)

L'interfaccia utente è stata pensata per rendere l'esperienza un'alternativa rispetto ai soliti tavoli da gioco, l'intenzione è stata non solo di catapultare il giocatore in una realtà parallela, ma anche quella di continuare a giocare (responsabilmente) in un ambiente insolito, ma divertente rispetto alle versioni tradizionali. Il tutto ha inizio sin dall'interfaccia del menù, dove si riconosce l'entità del gioco da tavolo dallo sfondo, confermato poi durante le partite. Ecco qualche esempio:





Statistiche Finali Simulazione					
Partite giocate: 50					
Bot	Vittorie	Punteggio Medio	Carte Pescate	Challenge	Challenge Vinte
ConservativeBot	19	0.0	338	0	0
RandomBot	27	0.0	367	0	0
Chiudi					

9. SCELTE PROGETTUALI E LIMITI NOTI DEL PROGETTO

Nel corso dello sviluppo abbiamo fatto alcune scelte non richieste o non indicate nella consegna finale del progetto:

- Effetto reverse in due giocatori: quando la partita si svolge tra due soli giocatori la carta Reverse ha lo stesso effetto di uno Skip, saltando il turno dell'avversario.
- Timer per UNO: Per dare un senso anche alla tempestività di gioco, abbiamo inserito due timer: 1 secondo per il giocatore di turno per chiamare 'UNO', poi 3 secondi in cui gli avversari possono premere il tasto 'CONTESTA'.

Come già riportato nel file [README.md](#) ci son state alcune limitazioni alle quali non siamo riusciti a rimediare per tempo:

- I bot non contestano autonomamente la mancata dichiarazione di UNO da parte del giocatore umano, gli unici a poter contestare sono i giocatori umani manualmente.
- Quando giocano un Wild, i bot selezionano il colore in maniera del tutto casuale, poiché la scelta sarebbe dovuta variare anche in base alla strategia, abbiamo deciso di adottare questa scelta in maniera completamente casuale.

Florea Alexandru Cosmin (M.2241398)
Giorgini Massimo (M.2234123)